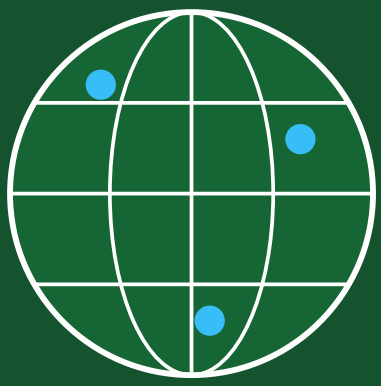

Web Development Fundamentals

How the web works, backends, APIs, databases, security and deployment

Request • Respond • Store • Secure • Deploy

Updated October 2026

Contents

1. How the web works
2. HTTP in depth
3. Backend basics
4. APIs: REST, GraphQL and WebSockets
5. Databases for the web
6. Authentication and authorization
7. Web security essentials
8. Domains, hosting and deployment
9. SEO and performance basics
10. Full-stack roadmap and launch checklist

> **About this guide**
> The earlier Frontend guide covers HTML, CSS, JavaScript and frameworks. This guide covers the rest of the web: how the pieces communicate, what runs on the server, how data is stored and how a site goes live safely.

The big picture

A web application has three main parts that talk to each other over the internet:

Layer	Responsibility	Examples
Client (frontend)	What users see and click	Browser, HTML/CSS/JS, React
Server (backend)	Business logic, security, talking to the database	Node.js, Python, .NET, Java
Data layer	Stores information permanently	PostgreSQL, MySQL, MongoDB, Redis

1. How the web works

The web works on a **request and response** cycle. A client asks for something; a server answers.

What happens when you type a URL

- **1. URL parsing:** the browser splits the address into protocol, host, path and query.
- **2. DNS lookup:** the host name (example.com) is translated into an IP address.
- **3. Connection:** the browser opens a TCP connection and, for HTTPS, a TLS handshake that encrypts traffic.
- **4. HTTP request:** the browser sends a request such as `GET /products`.
- **5. Server processing:** the server runs code, may query a database, and builds a response.
- **6. HTTP response:** the server sends back a status code, headers and a body (HTML, JSON, image).
- **7. Rendering:** the browser builds the page, then fetches linked CSS, JavaScript and images.

Anatomy of a URL

```
https://www.example.com:443/shop/shoes?size=42&color=red#reviews
\___/   \_____________/ \_/ \_________/ \_____________/ \_____/
scheme       host       port    path          query       fragment
```

Part	Meaning
Scheme	Protocol to use: `https` (secure) or `http`
Host	The server's name, resolved by DNS
Port	Network door on the server (443 for HTTPS, 80 for HTTP; usually hidden)
Path	Which resource on the server
Query	Extra parameters as key=value pairs
Fragment	A position within the page; never sent to the server

See it yourself
Open your browser's DevTools (F12), choose the **Network** tab, and reload a page. Every request and response described above is listed there.

2. HTTP in depth

HTTP is the language clients and servers use to talk. It is text-based, stateless (each request is independent) and built around methods, status codes and headers.

Common methods

Method	Purpose	Safe to repeat?
GET	Read a resource	Yes
POST	Create a resource or submit data	No
PUT	Replace a resource completely	Yes
PATCH	Change part of a resource	Usually
DELETE	Remove a resource	Yes

Status codes

Range	Meaning	Common codes
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 302 Found, 304 Not Modified
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx	Server error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Important headers

Header	Purpose
`Content-Type`	Format of the body, such as `application/json` or `text/html`
`Authorization`	Credentials, for example `Bearer <token>`
`Cache-Control`	How long responses may be cached
`Set-Cookie / Cookie`	Server stores and browser returns small pieces of data
`Accept`	Which formats the client can handle
`Origin / CORS headers`	Control which websites may call an API from a browser

Try requests with curl (in PowerShell use curl.exe)

```
curl -i https://example.com                    # show response headers
curl https://api.example.com/todos             # GET
curl -X POST http://localhost:3000/api/todos \
     -H "Content-Type: application/json" \
     -d '{"title":"Learn HTTP"}'               # POST JSON
```

3. Backend basics

The backend receives requests, applies rules, talks to databases and other services, and returns responses. You can write it in almost any language.

Stack	Language	Frameworks
Node.js	JavaScript / TypeScript	Express, Fastify, NestJS, Hono
Python	Python	FastAPI, Django, Flask
.NET	C#	ASP.NET Core (Minimal APIs, MVC)
Java / Kotlin	Java, Kotlin	Spring Boot, Ktor
PHP	PHP	Laravel, Symfony
Go / Rust	Go, Rust	Gin, Echo / Axum, Actix

A small API with Node.js and Express

Create a folder, run `npm init -y`, add `"type": "module"` to `package.json`, run `npm install express`, and save this as `server.js`:

```
import express from "express";

const app = express();
app.use(express.json());              // parse JSON request bodies

const todos = [];

app.get("/api/todos", (req, res) => {
  res.json(todos);
});

app.post("/api/todos", (req, res) => {
  const { title } = req.body;
  if (!title) {
    return res.status(400).json({ error: "title is required" });
  }
  const todo = { id: todos.length + 1, title, done: false };
  todos.push(todo);
  res.status(201).json(todo);
});

app.listen(3000, () => console.log("Running on http://localhost:3000"));
```

Start it with `node server.js` and test it with the curl commands from the previous page.

What a typical backend handles

- **Routing and validation:** match URLs and methods to code, and never trust input.
- **Business logic and data access:** apply your rules and read or write the database.
- **Errors, logging and configuration:** clear errors, useful logs, and settings and secrets in environment variables.

4. APIs: REST, GraphQL and WebSockets

An **API** (application programming interface) is how programs talk to your backend. On the web, data usually travels as **JSON**.

Style	How it works	Best for
REST	Resources identified by URLs, manipulated with HTTP methods	Most web and mobile back ends; simple and widely understood
GraphQL	One endpoint; the client asks for exactly the fields it needs	Complex data needs, many client types
WebSockets	A persistent two-way connection	Chat, live dashboards, multiplayer features
Server-Sent Events	Server pushes updates over one connection	Live feeds and notifications
gRPC	Compact binary calls between services	Internal service-to-service communication

REST design rules

Rule	Good example	Avoid
Use nouns for resources	`GET /users/42`	`GET /getUser?id=42`
Use plural collections	`/orders`	`/order`
Nest relationships sparingly	`GET /users/42/orders`	Deep nesting of many levels
Use the right method	`DELETE /orders/7`	`POST /deleteOrder`
Return correct status codes	`201` after creating, `404` if missing	`200` with an error message
Version your API	`/v1/products`	Breaking changes without warning
Paginate lists	`?page=2&limit=20`	Returning thousands of rows

A consistent error format

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

{
  "error": "validation_failed",
  "message": "Email address is not valid",
  "field": "email"
}
```

Document your API
Describe it with OpenAPI and publish interactive docs (Swagger UI, Redoc). See the Software Documentation guide for details.

5. Databases for the web

Almost every real web app stores data. Choose the database type that fits the shape of your data.

Type	Examples	Strengths
Relational (SQL)	PostgreSQL, MySQL, SQL Server, SQLite	Structured data, relationships, transactions, strong consistency
Document (NoSQL)	MongoDB, Firestore	Flexible JSON-like documents, fast iteration
Key-value / cache	Redis, Memcached	Very fast lookups, caching, sessions, rate limiting
Search	Elasticsearch, OpenSearch, Meilisearch	Full-text search and filtering
Object storage	Amazon S3, Azure Blob, Cloudflare R2	Files, images, videos, backups

SQL essentials

```
CREATE TABLE users (
  id          INTEGER PRIMARY KEY,
  email       TEXT NOT NULL UNIQUE,
  name        TEXT NOT NULL
);

CREATE TABLE orders (
  id        INTEGER PRIMARY KEY,
  user_id   INTEGER NOT NULL REFERENCES users(id),
  total     NUMERIC NOT NULL
);

INSERT INTO users (email, name) VALUES ('ada@example.com', 'Ada');

SELECT u.name, SUM(o.total) AS spent
FROM users u
JOIN orders o ON o.user_id = u.id
GROUP BY u.name
ORDER BY spent DESC;
```

Good practices

- Use **primary keys** and **foreign keys** to keep data consistent.
- Add **indexes** on columns you search or join on often; they speed up reads but slow writes slightly.
- Use **transactions** when several changes must succeed or fail together.
- Manage schema changes with **migrations** stored in Git.
- Use an **ORM** (Prisma, Drizzle, SQLAlchemy, Entity Framework Core) for convenience, but learn SQL too.
- Take **backups** and test restoring them.
- Start with **PostgreSQL** unless you have a clear reason for something else.

6. Authentication and authorization

Authentication answers "who are you?". **Authorization** answers "what are you allowed to do?". Confusing them is a common source of security bugs.

Sessions vs tokens

Approach	How it works	Notes
Server sessions + cookies	The server stores session data and gives the browser a session ID in a cookie	Simple and secure for traditional web apps; easy to revoke
JWT / access tokens	The client sends a signed token in the Authorization header	Common for APIs and mobile apps; keep lifetimes short
OAuth 2.0 / OpenID Connect	Users sign in through a provider such as Google, Microsoft or GitHub	Avoids handling passwords yourself
Passkeys (WebAuthn)	Phishing-resistant sign-in using device credentials	Increasingly supported; a strong modern option

Cookie settings that matter

Flag	Effect
`HttpOnly`	JavaScript cannot read the cookie, which limits damage from XSS
`Secure`	Sent only over HTTPS
`SameSite=Lax / Strict`	Limits cross-site sending, helping to prevent CSRF

Password rules for developers

- Never store plain-text passwords. Hash them with a slow algorithm such as **Argon2**, **bcrypt** or **scrypt**, with a unique salt.
- Prefer a trusted **identity provider or auth library** over building your own login system.
- Offer **multi-factor authentication**.
- Rate-limit login attempts and give generic error messages.
- Check **authorization on the server** for every request. Hiding a button in the UI is not security.

Roles and permissions

Define roles (for example admin, editor, viewer) and check them in the backend. Apply the **principle of least privilege**: give each user and service only the access it needs.

7. Web security essentials

Treat all input as hostile and assume that anything running in the browser can be inspected and altered by the user. The **OWASP Top 10** lists the most critical web risks; check the latest edition at owasp.org.

Threat	What it is	Main defense
SQL injection	Attacker input becomes part of a database query	Parameterized queries or an ORM
XSS (cross-site scripting)	Malicious script runs in another user's browser	Escape output, framework defaults, Content-Security-Policy
CSRF	A malicious site makes the browser send unwanted requests	SameSite cookies, CSRF tokens
Broken access control	Users reach data or actions they should not	Server-side permission checks on every request
Insecure configuration	Default passwords, debug mode on, open storage	Hardened defaults, review, automated scans
Vulnerable dependencies	Known flaws in libraries you use	Audit tools, regular updates
Sensitive data exposure	Secrets or personal data leaked	HTTPS, encryption, secret managers, minimal logging
SSRF	Server tricked into requesting internal resources	Validate URLs, restrict outbound access

Prevent SQL injection

```
# DANGEROUS: user input is pasted into the query
cur.execute(f"SELECT * FROM users WHERE email = '{email}'")

# SAFE: the database treats the value strictly as data
cur.execute("SELECT * FROM users WHERE email = ?", (email,))
```

Security headers and basics

- `Strict-Transport-Security` (HSTS): force HTTPS.
- `Content-Security-Policy`: restrict where scripts and resources may load from.
- `X-Content-Type-Options: nosniff` and a safe `Referrer-Policy`.
- Configure **CORS** to allow only the origins you trust, never a blanket wildcard for authenticated APIs.
- Keep secrets in environment variables or a secrets manager, never in Git or frontend code.
- Scan for weaknesses before launch with free tools such as **OWASP ZAP**.

8. Domains, hosting and deployment

Domains and DNS

A **domain name** is bought from a registrar (for example Namecheap, Cloudflare, Google Domains successors, GoDaddy). **DNS records** tell the internet where to find your services.

Record	Purpose	Example
A	Maps a name to an IPv4 address	example.com to 203.0.113.10
AAAA	Maps a name to an IPv6 address	example.com to 2001:db8::10
CNAME	Alias of one name to another	www to example.com
MX	Where to deliver email	mail provider servers
TXT	Text data: email authentication (SPF, DKIM), verification	v=spf1 ...

Hosting options

Option	Examples	Best for
Static hosting + CDN	Netlify, Vercel, Cloudflare Pages, GitHub Pages	Frontend-only sites, documentation, portfolios
Platform as a Service	Render, Railway, Fly.io, Azure App Service, Heroku	Full apps without managing servers
Containers	AWS ECS / Fargate, Google Cloud Run, Azure Container Apps	Docker-based apps that scale
Serverless functions	AWS Lambda, Azure Functions, Cloudflare Workers	Event-driven code, pay per use
Virtual server (VPS)	DigitalOcean, Hetzner, Linode, EC2	Full control; you manage updates and security

A simple deployment pipeline

- Push code to Git (GitHub or GitLab).
- CI runs tests, linting and the build.
- On success, CI builds a Docker image or static bundle and deploys it to staging.
- After checks, promote to production; keep a rollback plan.
- Use **HTTPS** everywhere; free certificates come from Let's Encrypt and most hosts enable them automatically.
- Add **monitoring**: uptime checks, error tracking (for example Sentry) and logs.

Environments

Keep **development**, **staging** and **production** separate, each with its own database and secrets. Configure them through environment variables, following the twelve-factor app approach.

9. SEO and performance basics

SEO (search engine optimization) helps people find your site through search. Most of it comes from good fundamentals.

On-page essentials

- A unique, descriptive `<title>` (about 50 to 60 characters) and a `meta description` for each page.
- One `<h1>` per page and a logical heading structure.
- Semantic HTML, descriptive link text and `alt` text on images.
- Clean, readable URLs, for example `/guides/web-security`.
- Mobile-friendly, responsive layouts and fast loading.
- Open Graph tags so links look good when shared on social media.

Technical SEO

Item	Purpose
`sitemap.xml`	Lists the pages you want search engines to find
`robots.txt`	Tells crawlers which paths they may or may not visit
Canonical URL	Tells search engines which version of duplicate content is the main one
Structured data (JSON-LD)	Helps search engines understand content such as articles and products
Server-side rendering / static generation	Makes sure crawlers receive full HTML, not an empty page
Redirects (301)	Preserve rankings when URLs change

Performance on the server side

- Enable **compression** (gzip or Brotli) and **HTTP caching** with sensible Cache-Control headers.
- Serve static files through a **CDN**, close to your users.
- Cache expensive queries, for example in Redis.
- Add **database indexes** and avoid fetching more data than needed.
- Use **HTTP/2 or HTTP/3**, which most hosts support by default.

> **Measure**
> Use **Lighthouse**, PageSpeed Insights and Google Search Console to find real problems, and keep an eye on Core Web Vitals over time.

10. Full-stack roadmap and launch checklist

Suggested learning path

Stage	Learn	Build
1. Frontend basics	HTML, CSS, JavaScript, Git	Static portfolio site
2. How the web works	HTTP, DNS, DevTools, curl	Inspect and debug real sites
3. A backend language	Node.js, Python or C#; routing, validation	Simple REST API
4. Databases	SQL, PostgreSQL, migrations, an ORM	API with persistent data
5. Auth and security	Sessions, OAuth, OWASP basics	Login and user accounts
6. Deployment	Docker, CI/CD, hosting, monitoring	Live, public full-stack app

Project ideas

- **Starter:** link shortener, notes app, URL-based guestbook.
- **Intermediate:** blog with accounts and comments, task manager, expense tracker with charts.
- **Advanced:** e-commerce store with payments, real-time chat, multi-tenant SaaS dashboard.

Launch checklist

- [ ] HTTPS is enabled and HTTP redirects to HTTPS
- [ ] All secrets are in environment variables; none in Git
- [ ] Input is validated and queries are parameterized
- [ ] Authorization is checked on the server
- [ ] CORS, cookies and security headers are configured
- [ ] Database backups run and a restore has been tested
- [ ] Errors are logged and monitored; uptime alerts are set
- [ ] Pages have titles, descriptions, sitemap and robots.txt
- [ ] Performance and accessibility checked with Lighthouse
- [ ] A rollback plan exists for bad deployments

Free learning resources

- **MDN Web Docs** (developer.mozilla.org): HTTP, web APIs and server-side guides.
- **web.dev**: performance, security and modern web practices.
- **OWASP** (owasp.org): security guides and cheat sheets.
- **The Odin Project** and **freeCodeCamp**: full-stack curriculums.

> **Final advice**
> Understanding how requests, data and security fit together matters more than any single framework. Build a small full-stack project end to end, deploy it, and learn from what breaks.